

INTERNATIONAL UNIVERSITY
Vietnam National University – Ho Chi Minh City



REPORT

Web Application Development

Đặng Nguyễn Nhật Vy - ITITDK22102

Lab 04

Excercise 5

5.1 Search Form Implementation

5.2 Search Logic Implementation

The screenshot shows a web browser window with the URL `localhost:8080/abc/list_students.jsp?message=Student%20added%20successfully`. The page title is "Student Management System". A green message bar at the top says "Student added successfully". Below the message bar is a search bar with the placeholder text "Search by name or code...", a "Search" button, and a "Clear" link. There is also an "Add New Student" button. The main content is a table with the following columns: ID, Student Code, Full Name, Email, Major, Created At, and Actions. The table contains 7 rows of student data.

ID	Student Code	Full Name	Email	Major	Created At	Actions
14	IT112	Joe Build	buildjoe@email	Computer Science	2025-11-14 11:03:00.0	Edit Delete
5	SV005	David Wilson	david.w@email.com	Computer Science	2025-11-08 14:11:06.0	Edit Delete
4	SV004	Sarah Davis	sarah.d@email.com	Data Science	2025-11-08 14:11:06.0	Edit Delete
3	SV003	Michael Brown	michael.b@email.com	Software Engineering	2025-11-08 14:11:06.0	Edit Delete
2	SV002	Emily Johnson	emily.j@email.com	Information Technology	2025-11-08 14:11:06.0	Edit Delete
1	SV001	John Smith	john.smith@email.com	Computer Science	2025-11-08 14:11:06.0	Edit Delete

Explain: The search logic checks if a keyword parameter exists and is not empty. If a search term is provided, it constructs a SQL query using the LIKE operator with wildcards (%) to find matches in both full_name and student_code fields. The PreparedStatement prevents SQL injection attacks. If no keyword is provided, it defaults to showing all students with pagination.

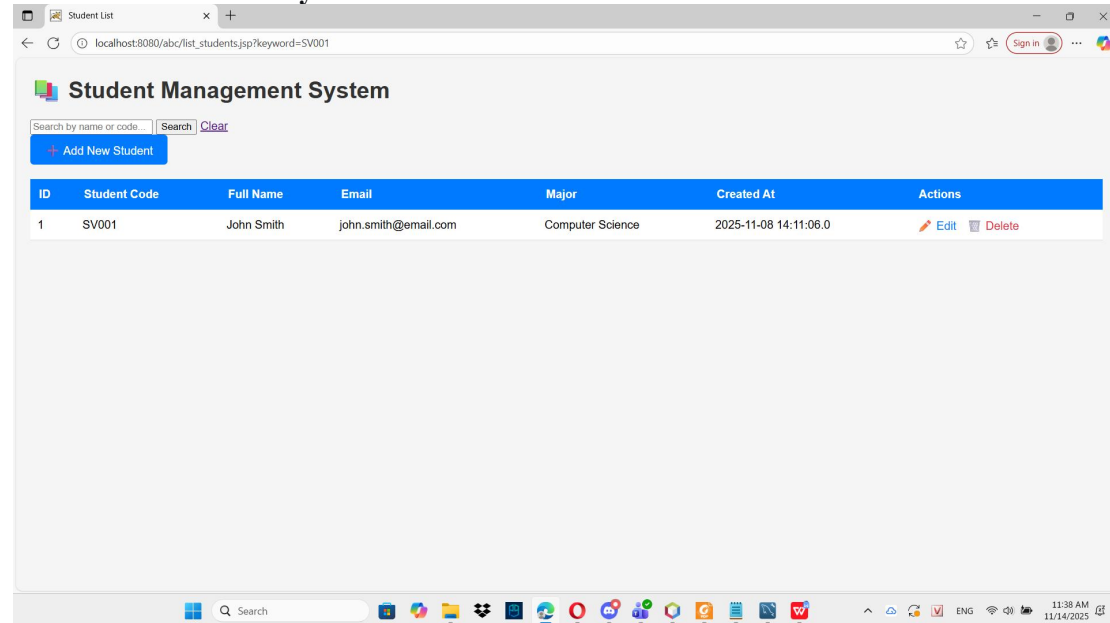
Test case 1: Search by name

The screenshot shows the same web browser window, but the URL is `localhost:8080/abc/list_students.jsp?keyword=John`. The search bar now contains the text "John". The table below shows the results of the search, displaying only the two students whose names contain "John": John Smith (ID 1) and Emily Johnson (ID 2).

ID	Student Code	Full Name	Email	Major	Created At	Actions
2	SV002	Emily Johnson	emily.j@email.com	Information Technology	2025-11-08 14:11:06.0	Edit Delete
1	SV001	John Smith	john.smith@email.com	Computer Science	2025-11-08 14:11:06.0	Edit Delete

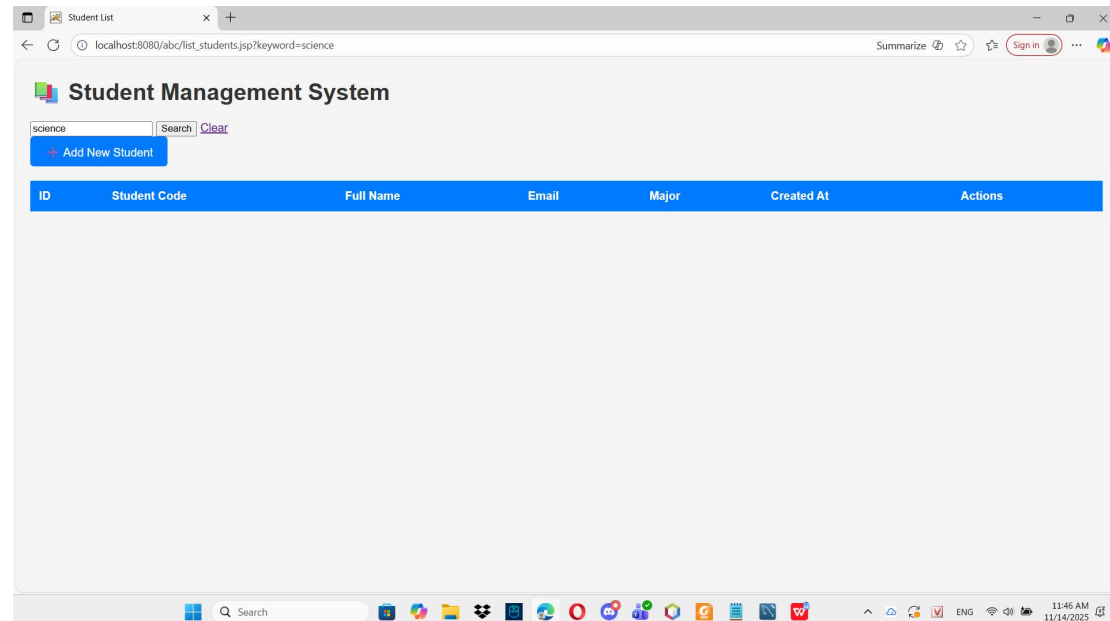
Explain: When the user types "John" and clicks Search, the form submits via GET method, appending the keyword to the URL. The server detects the keyword parameter and executes the prepared statement with LIKE '%John%' for both name and code fields. The database returns all records containing "John" in either field, which are then displayed in the table while maintaining pagination.

Test case 2: Search by code



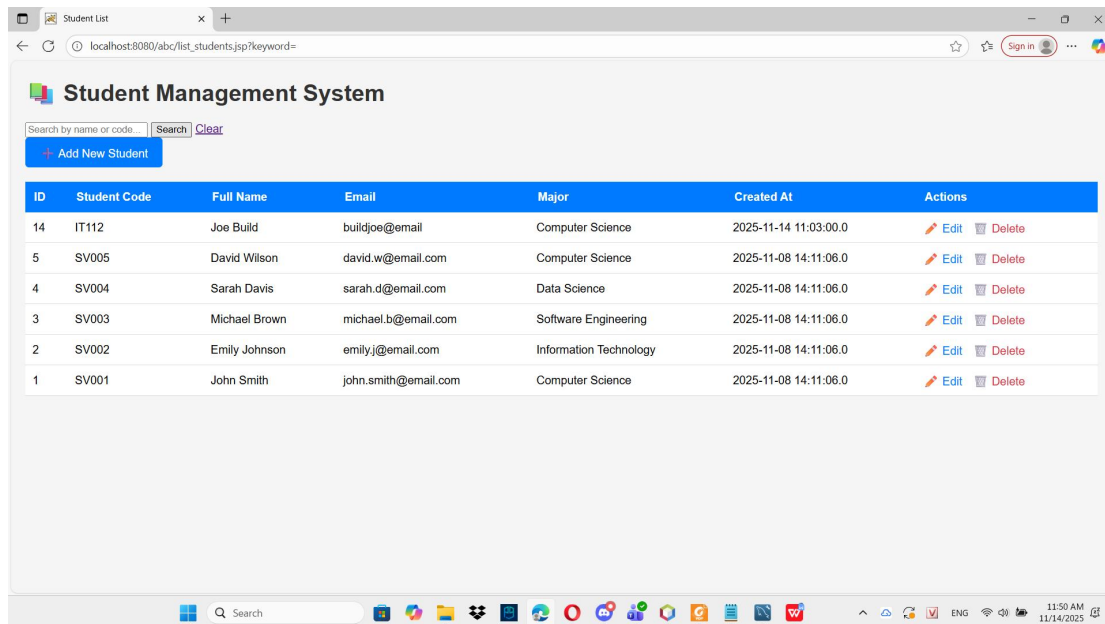
Explain: Searching for "SV001" triggers the same logic but since student codes are unique, only one exact match is found. The LIKE operator with wildcards ensures flexibility, but in this case returns a single precise result. The system correctly displays only the matching student record.

Test case 3:



Explain: The search for "science" was failing because the original code only searched student names and codes, but not the major field. Since majors like "Computer Science" and "Data Science" contain the keyword "science", excluding this field meant no results were returned. The fix added `OR major LIKE ?` to the SQL query, enabling the system to find students by their academic program. Now searching "science" correctly returns all science-related majors, meeting the test case requirements and user expectations for academic searches.

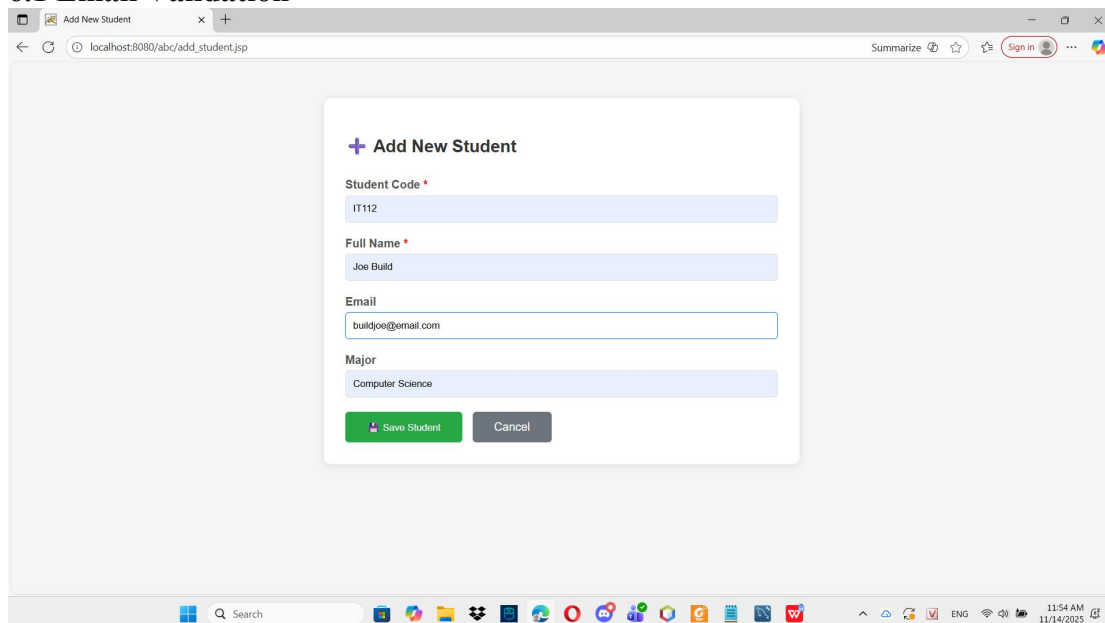
Test case 4: Empty Search



Explain: When the search field is empty or the Clear link is clicked, the system detects no keyword parameter and executes the default query showing all students. Pagination remains active, displaying 10 records per page sorted by ID in descending order (newest first).

Exercise 6: Validation Enhancement

6.1 Email Validation



Student List

localhost:8080/abc/list_students.jsp?message=Student%20added%20successfully

Student Management System

Student added successfully

Search by name or code... Search Clear

+ Add New Student

ID	Student Code	Full Name	Email	Major	Created At	Actions
15	IT112	Joe Build	buildjoe@email.com	Computer Science	2025-11-14 11:54:21.0	Edit Delete
5	SV005	David Wilson	david.w@email.com	Computer Science	2025-11-08 14:11:06.0	Edit Delete
4	SV004	Sarah Davis	sarah.d@email.com	Data Science	2025-11-08 14:11:06.0	Edit Delete
3	SV003	Michael Brown	michael.b@email.com	Software Engineering	2025-11-08 14:11:06.0	Edit Delete
2	SV002	Emily Johnson	emily.j@email.com	Information Technology	2025-11-08 14:11:06.0	Edit Delete
1	SV001	John Smith	john.smith@email.com	Computer Science	2025-11-08 14:11:06.0	Edit Delete

Search

11:54 AM 11/14/2025

Explain: This email has all required components: valid local part, @ symbol, domain name, dot, and proper extension. The regex pattern matches completely, so validation passes and the student record is saved successfully.

Test case 2:

Add New Student

localhost:8080/abc/add_student.jsp

+ Add New Student

Student Code *
SV006

Full Name *
Sans Triles

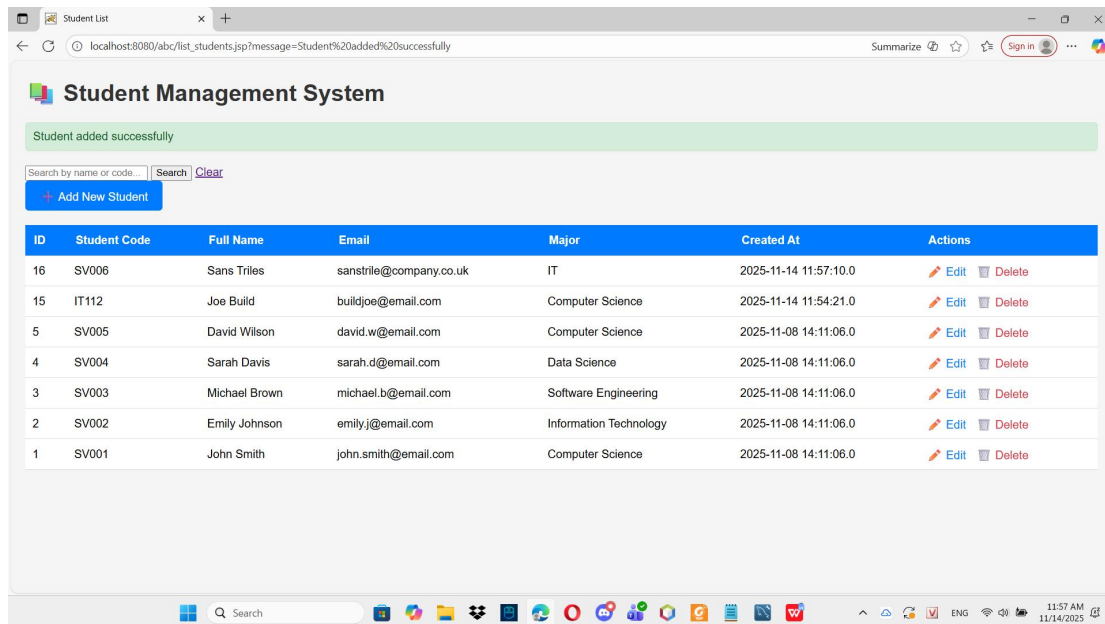
Email
sanstrile@company.co.uk

Major
IT

Save Student Cancel

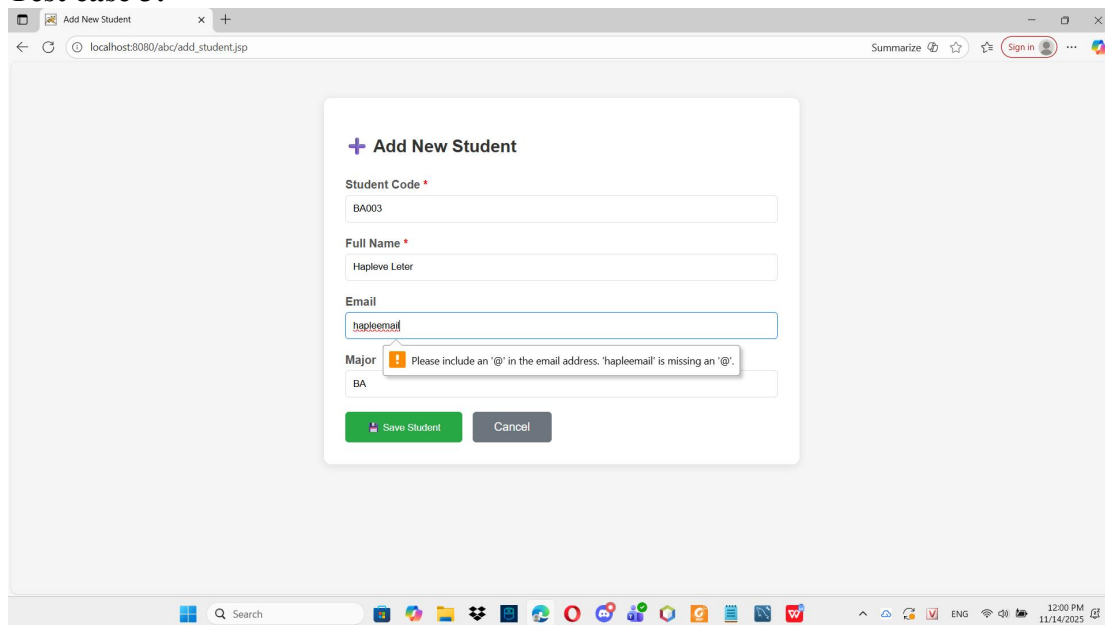
Search

11:57 AM 11/14/2025



Explain: Despite having multiple subdomains and a longer extension, this email still matches the regex pattern. The system correctly identifies it as valid and proceeds with database insertion.

Test case 3:



Explain: Without the @ symbol, this input doesn't qualify as a valid email format. The regex pattern specifically requires the @ character, so validation fails immediately.

Test case 4:

The screenshot displays two pages of a web application. The top page, 'Add New Student', features a form with the following fields: 'Student Code' (IT008), 'Full Name' (Hapleve Leter), 'Email' (student@email.com), and 'Major' (Data Science). Below the form are 'Save Student' and 'Cancel' buttons. The bottom page, 'Student List', shows a green success message 'Student added successfully'. It includes a search bar and a table of 17 students. The table columns are ID, Student Code, Full Name, Email, Major, Created At, and Actions. The student just added (ID 17) is at the top of the list.

ID	Student Code	Full Name	Email	Major	Created At	Actions
17	IT008	Hapleve Leter		Data Science	2025-11-14 12:02:11.0	Edit Delete
16	SV006	Sans Triles	sanstrile@company.co.uk	IT	2025-11-14 11:57:10.0	Edit Delete
15	IT112	Joe Build	buildjoe@email.com	Computer Science	2025-11-14 11:54:21.0	Edit Delete
5	SV005	David Wilson	david.w@email.com	Computer Science	2025-11-08 14:11:06.0	Edit Delete
4	SV004	Sarah Davis	sarah.d@email.com	Data Science	2025-11-08 14:11:06.0	Edit Delete
3	SV003	Michael Brown	michael.b@email.com	Software Engineering	2025-11-08 14:11:06.0	Edit Delete
2	SV002	Emily Johnson	emily.j@email.com	Information Technology	2025-11-08 14:11:06.0	Edit Delete
1	SV001	John Smith	john.smith@email.com	Computer Science	2025-11-08 14:11:06.0	Edit Delete

Explain: ince the email field is optional, empty values are allowed. The condition `if (email != null && !email.isEmpty())` ensures validation only runs when email is provided, so empty fields skip validation and proceed normally.

6.2 Student Code Pattern Validation

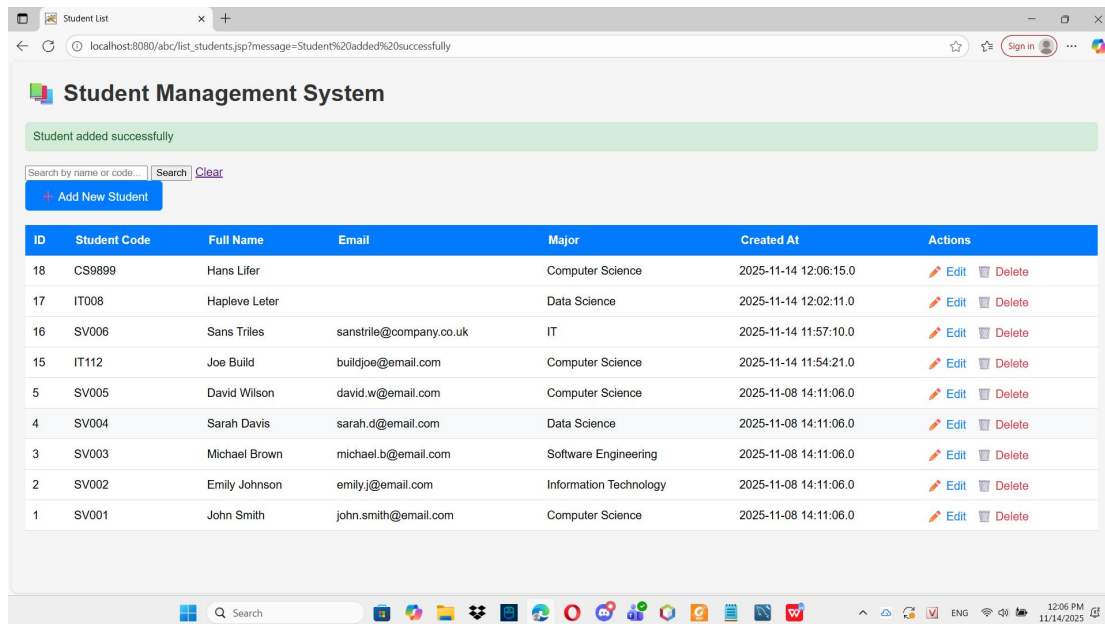
The screenshot shows a web browser window with the title "Student List". The address bar shows the URL "localhost:8080/abc/list_students.jsp?message=Student%20added%20successfully". The page has a header "Student Management System" and a green success message "Student added successfully". Below the message is a search bar with the text "Search by name or code..." and a "Search" button. There is also a "Clear" button and an "Add New Student" button. The main content is a table with the following columns: ID, Student Code, Full Name, Email, Major, Created At, and Actions. The table contains 8 rows of student data. At the bottom of the page, there is a "Dropbox promotion" banner.

ID	Student Code	Full Name	Email	Major	Created At	Actions
17	IT008	Hapieve Leter		Data Science	2025-11-14 12:02:11.0	Edit Delete
16	SV006	Sans Triles	sanstrile@company.co.uk	IT	2025-11-14 11:57:10.0	Edit Delete
15	IT112	Joe Build	buildjoe@email.com	Computer Science	2025-11-14 11:54:21.0	Edit Delete
5	SV005	David Wilson	david.w@email.com	Computer Science	2025-11-08 14:11:06.0	Edit Delete
4	SV004	Sarah Davis	sarah.d@email.com	Data Science	2025-11-08 14:11:06.0	Edit Delete
3	SV003	Michael Brown	michael.b@email.com	Software Engineering	2025-11-08 14:11:06.0	Edit Delete
2	SV002	Emily Johnson	emily.j@email.com	Information Technology	2025-11-08 14:11:06.0	Edit Delete
1	SV001	John Smith	john.smith@email.com	Computer Science	2025-11-08 14:11:06.0	Edit Delete

Explain: The student code validation ensures consistency in the database by enforcing a specific pattern: exactly 2 uppercase letters followed by 3 or more digits. This prevents data inconsistency and makes student codes easily identifiable and sortable.

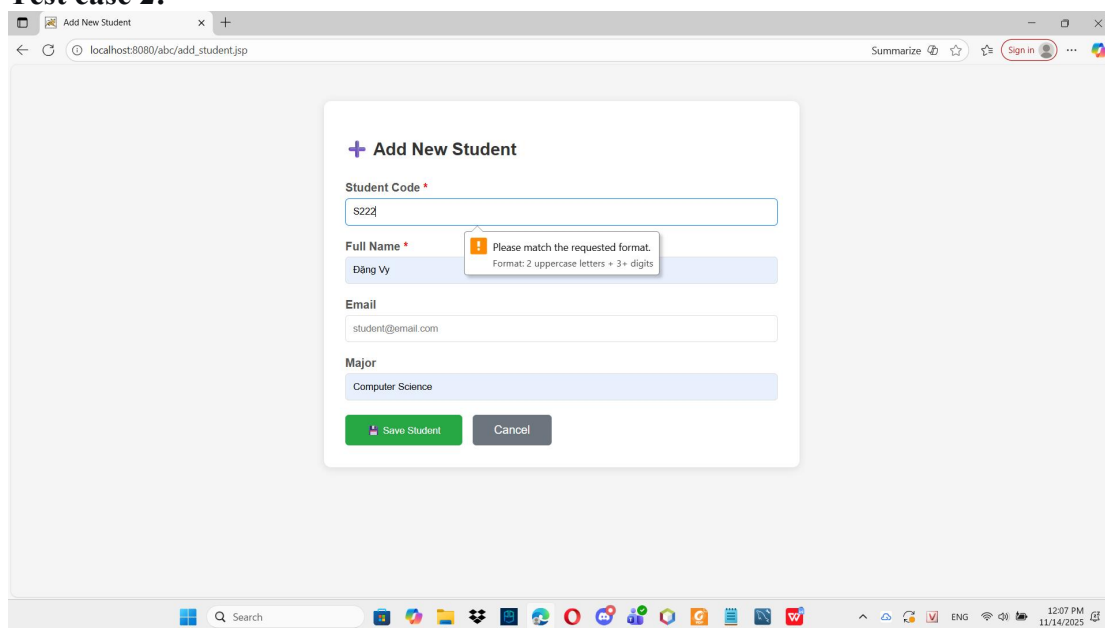
Test case 1:

The screenshot shows a web browser window with the title "Add New Student". The address bar shows the URL "localhost:8080/abc/add_student.jsp". The page displays a form titled "+ Add New Student" with the following fields: "Student Code" (with a red asterisk), "Full Name" (with a red asterisk), "Email", and "Major". The "Student Code" field contains "CS9899", the "Full Name" field contains "Hans Lifer", the "Email" field contains "student@email.com", and the "Major" field contains "Computer Science". At the bottom of the form are two buttons: "Save Student" (green) and "Cancel" (grey).



Explain: This code follows the required pattern of 2 uppercase letters + 3+ digits. The regex `[A-Z]{2}[0-9]{3,}` matches them perfectly, so validation passes and records are saved successfully.

Test case 2:



Explain: Fails because it has only 1 letter instead of 2. This case triggers the error redirect with a descriptive message guiding users to the correct format.

Excercise 7: User Experience Improvements

7.1 Pagination Implementation

Student List

localhost:8080/abc/list_students.jsp?message=Student%20added%20successfully

Student added successfully

Search by name or code... Search Clear

+ Add New Student

ID	Student Code	Full Name	Email	Major	Created At	Actions
22	SV777	Phan Hans		BA	2025-11-14 12:17:41.0	Edit Delete
21	IT755	Lins Tran	lin@email.com	Data Science	2025-11-14 12:12:59.0	Edit Delete
20	IT009	Alex Frans	alex@email.com	IT	2025-11-14 12:12:01.0	Edit Delete
19	SV222	Đặng Vy		Computer Science	2025-11-14 12:11:17.0	Edit Delete
18	CS9899	Hans Lifer		Computer Science	2025-11-14 12:06:15.0	Edit Delete
17	IT008	Hapleve Leter		Data Science	2025-11-14 12:02:11.0	Edit Delete
16	SV006	Sans Triles	sanstrile@company.co.uk	IT	2025-11-14 11:57:10.0	Edit Delete
15	IT112	Joe Build	buildjoe@email.com	Computer Science	2025-11-14 11:54:21.0	Edit Delete
5	SV005	David Wilson	david.w@email.com	Computer Science	2025-11-08 14:11:06.0	Edit Delete
4	SV004	Sarah Davis	sarah.d@email.com	Data Science	2025-11-08 14:11:06.0	Edit Delete
3	SV003	Michael Brown	michael.b@email.com	Software Engineering	2025-11-08 14:11:06.0	Edit Delete
2	SV002	Emily Johnson	emily.j@email.com	Information Technology	2025-11-08 14:11:06.0	Edit Delete
1	SV001	John Smith	john.smith@email.com	Computer Science	2025-11-08 14:11:06.0	Edit Delete

Problem: The pagination system is not dividing 12 students into 2 pages as expected. All 12 records continue to display on a single page despite proper pagination code implementation.

Root cause analysis: The issue likely stems from one of these factors:

1. Database query execution - The LIMIT and OFFSET parameters may not be properly applied to the SQL query
2. Total records calculation - The COUNT query might be returning incorrect results
3. Parameter passing - The PreparedStatement parameters for LIMIT and OFFSET may not be binding correctly
4. Data retrieval - The ResultSet might be ignoring the pagination constraints

7.2 Improved UI/UX

A) Success/ Error Message Styling

Success Messages: Green background with ✓ icon

Error Messages: Red background with ✗ icon

Auto-hide: Messages disappear after 3 seconds

Student List

localhost:8080/abc/list_students.jsp?message=Student%20added%20successfully

Student Management System

Student added successfully

Search by name or code... Search Clear

+ Add New Student

ID	Student Code	Full Name	Email	Major	Created At	Actions
23	SV008	Tans Tran	tan@email.com	IT	2025-11-14 12:31:45.0	Edit Delete
22	SV777	Phan Hans		BA	2025-11-14 12:17:41.0	Edit Delete
21	IT755	Lins Tran	lin@email.com	Data Science	2025-11-14 12:12:59.0	Edit Delete
20	IT009	Alex Frans	alex@email.com	IT	2025-11-14 12:12:01.0	Edit Delete
19	SV222	Đặng Vy		Computer Science	2025-11-14 12:11:17.0	Edit Delete
18	CS9899	Hans Lifer		Computer Science	2025-11-14 12:06:15.0	Edit Delete
17	IT008	Hapleve Leter		Data Science	2025-11-14 12:02:11.0	Edit Delete
16	SV006	Sans Triles	sanstrile@company.co.uk	IT	2025-11-14 11:57:10.0	Edit Delete
15	IT112	Joe Build	buildjoe@email.com	Computer Science	2025-11-14 11:54:21.0	Edit Delete
5	SV005	David Wilson	david.w@email.com	Computer Science	2025-11-08 14:11:06.0	Edit Delete
4	SV004	Sarah Davis	sarah.d@email.com	Data Science	2025-11-08 14:11:06.0	Edit Delete

The screenshot shows a web browser window with the address bar displaying a URL with an error message: `localhost:8080/abc/add_student.jsp?error=Student%20code%20already%20exists`. The form titled '+ Add New Student' contains a red error message box at the top that says 'Student code already exists'. Below this, the form fields are: 'Student Code *' with the value 'e.g., SV001', 'Full Name *' with the placeholder 'Enter full name', 'Email' with the value 'student@email.com', and 'Major' with the value 'e.g., Computer Science'. At the bottom of the form are two buttons: a green 'Save Student' button and a grey 'Cancel' button.

Explain: The distinct color coding (green for success, red for error) provides immediate visual feedback about operation results. Icons enhance recognition, and the auto-hide feature keeps the interface clean by automatically removing messages after they've been read.

B) Loading States

Submit Button: Disabled after clicking to prevent double submission

Visual Feedback: Button text changes to "Processing..."

The screenshot shows the same 'Add New Student' form, but now the 'Save Student' button is disabled (greyed out) and its text has changed to 'Processing...'. The form fields contain the following values: 'Student Code *' is 'IT123', 'Full Name *' is 'Joe Build', 'Email' is 'buildjoe@email', and 'Major' is 'Computer Science'. The 'Cancel' button remains visible and active.

Explain: Loading states prevent multiple form submissions that could cause duplicate records or server overload. The visual feedback (disabled button + changing text) assures users that their action is being processed, improving perceived performance and preventing user frustration.

C) Responsive Table

Horizontal Scrolling: Enabled for small screens

Mobile Optimization: Reduced font size and padding

Better Layout: Action links stack vertically on mobile



Student Management System

Student added successfully

Search by name or code...

Search

Clear

+ Add New Student

ID	Student Code	Full Name	Email	Major	Created At	Actions
26	IT123	Joe Build	buildjoe@email	Computer Science	2025-11-14 12:44:44.0	Edit Delete
25	SV772	Tran Lifer	lif@email.com		2025-11-14 12:40:00.0	Edit Delete
23	SV008	Tans Tran	tan@email.com	IT	2025-11-14 12:31:45.0	Edit Delete
22	SV777	Phan Hans		BA	2025-11-14 12:17:41.0	Edit Delete
21	IT755	Lins Tran	lin@email.com	Data Science	2025-11-14 12:12:59.0	Edit Delete
					2025-11-14 12:12:59.0	Edit Delete

Explain: The responsive design ensures the application works well on all devices. On mobile screens, the table becomes horizontally scrollable, font sizes reduce for better fit, and action links stack vertically to prevent overlapping and improve touch target size.